

## Document de Spécification Technique

### Projet : AI Agent d'interrogation intelligente des données bancaires

#### 1. Objectif

Ce document décrit l'architecture technique de l'application, les composants, les technologies utilisées et le fonctionnement global du système.

#### 2. Architecture générale

Utilisateur → Interface Streamlit → Authentification → Analyse de la question → RAG → Prompt → LLM (Gemini) → Génération SQL → Validation SQL → SQLite → Résultat → Réponse

#### 3. Technologies utilisées

| Technologie             | Rôle                                           |
|-------------------------|------------------------------------------------|
| Python 3                | Langage principal                              |
| Streamlit               | Interface utilisateur                          |
| FastAPI                 | Backend API (logique métier +orchestration IA) |
| SQLite                  | Base de données                                |
| Pandas                  | Manipulation des données                       |
| SQLAlchemy              | Accès base de données                          |
| FAISS                   | Recherche vectorielle (RAG)                    |
| Sentence Transformers   | Embeddings texte                               |
| API Gemini              | Génération SQL                                 |
| OpenPyXL                | Lecture Excel                                  |
| LangChain               | Orchestration IA                               |
| Docker + Docker Compose | Conteneurisation et déploiement                |

## Document de Spécification Technique

### 4. Modules du système

| Module                | Description                               |
|-----------------------|-------------------------------------------|
| Interface (Streamlit) | Connexion, questions, affichage résultats |
| Authentification      | Gestion utilisateur et administrateur     |
| Analyse               | Nettoyage et compréhension de la question |
| Mapping               | Lecture fichiers Excel/JSON/CSV           |
| RAG                   | Récupération des données pertinentes      |
| LLM                   | Génération SQL via Gemini                 |
| Validation SQL        | Sécurité des requêtes                     |
| Base de données       | Stockage SQLite                           |

### 5. Sécurité

- Utilisateur limité à SELECT
- Admin autorisé sur toutes les requêtes
- Validation des requêtes SQL
- Journalisation des actions sensibles

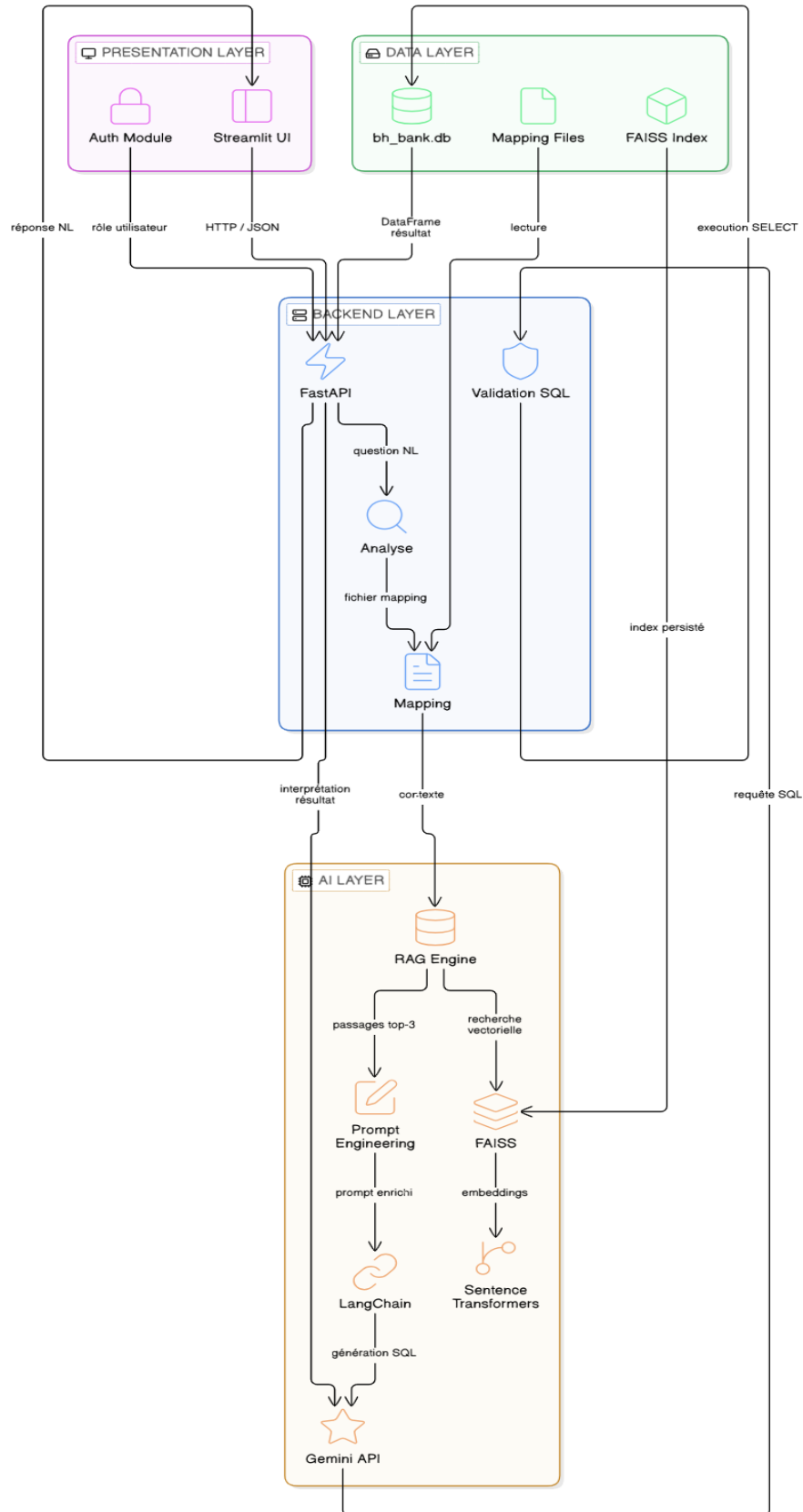
### 6. Base de données

Tables principales : CUSTOMER, ACCOUNT, TRANSACTION, BALANCE\_HISTORY, BRANCH, CARD

### 7. Flux de traitement

1. Utilisateur pose une question
2. Analyse de la question
3. Récupération via RAG
4. Génération du prompt
5. Génération SQL par Gemini
6. Validation SQL
7. Exécution SQLite
8. Réponse affichée

## Document de Spécification Technique



## Document de Spécification Technique

Le système est organisé en 4 couches distinctes :

La **couche Présentation** (Streamlit) constitue l'interface utilisateur : saisie de la question en langage naturel, upload du fichier de mapping et affichage de la réponse. Un module d'authentification gère les rôles (utilisateur / administrateur).

La **couche Données** regroupe les trois éléments de persistance : la base SQLite bh\_bank.db contenant les 6 tables bancaires, les fichiers de mapping (Excel / JSON / CSV) décrivant les correspondances métier, et l'index FAISS stocké sur disque contenant le schéma vectorisé.

La **couche Backend** (FastAPI) joue le rôle de coordinateur central. Elle orchestre séquentiellement les modules Analyse (extraction des mots-clés), Mapping (traduction des termes métier), Validation SQL (sécurité et contrôle des droits) et formatage de la réponse finale.

La **couche IA** constitue le cœur intelligent du système. Le moteur RAG utilise Sentence Transformers pour vectoriser la question, puis FAISS pour retrouver les 3 passages les plus pertinents du schéma. Ces passages alimentent le module de Prompt Engineering qui construit un prompt enrichi. LangChain orchestre l'appel à l'API Gemini pour générer la requête SQL, gère l'auto-correction en cas d'erreur (jusqu'à 3 tentatives), puis sollicite une seconde fois Gemini pour reformuler le résultat en langage naturel.